

Numerische Lineare Algebra: Iteration & Zufall

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- Komplexitätsanalyse / $O()$ -Notation
- Stabilität und Konditionierung
- Orthogonalmatrizen
- Spektralnorm $\| \cdot \|_2$
- LGS: Problem, Direkte Algorithmen
- **Lineare Algebra:**
 - Eigenwerte und Eigenvektoren
 - Ähnlichkeitstransformationen: $B = Q A Q^{-1}$
- **Wahrscheinlichkeitstheorie:** Zufallsvektoren
- **Analysis I:**
 - Konvergenz von Folgen
 - Geometrische Reihe $\sum_k p^k$ für $|p| < 1$

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $A \in \mathbb{R}^{n \times n}$ und $v \neq 0$ ein Eigenvektor mit $Av = \lambda v$. Welche Aussage gilt dann **immer**?

1. $A^k v = \lambda^k v$ für alle $k \in \mathbb{N}$
2. Es muss $\|v\| = 1$ gelten
3. $\lambda > 0$
4. v ist der einzige Eigenvektor zu λ

Frage 2: Sei $|p| < 1$. Was ist $\sum_{k=0}^{\infty} p^k$?

1. $\frac{1}{1-p}$
2. $\frac{p}{1-p}$
3. $\frac{1}{p-1}$
4. Die Reihe divergiert.

Überblick

- Numerische lineare Algebra ist ein sehr großes Feld.
- Wir haben bisher die wichtigsten Lösungsmethoden gesehen.
- Die Kernidee dabei war *Matrixzerlegung* (*LU*, *QR*, *Cholesky*, ...).
- Heute sehen wir in kürzerer Form zwei weitere wichtige Kernideen:
 - *Iterative Methoden*
 - verkürzen *Laufzeit* auf Kosten von Genauigkeit.
 - *Probabilistische Methoden*
 - ersetzen große Probleme durch kleinere mit *höchstwahrscheinlich* sehr ähnlicher Lösung.
- An beiden wird in der numerischen linearen Algebra aktiv geforscht.

Eigenwertprobleme

Lösungsverfahren für Eigenwertprobleme

- *Direkte Methoden existieren:*
 - Charakteristisches Polynom: $\det(\mathbf{A} - \lambda \mathbf{I}) = 0 \rightarrow$ Nullstellensuche
 - Problem:
Polynom vom Grad n für $n \times n$ Matrix $\rightarrow$ keine direkte Lösung für $n \geq 5$
 - Numerisch instabil und teuer: $O(n^4)$ oder schlechter
- *Wann sind direkte Methoden unzureichend?*
 - Große Matrizen ($n > 1000$): $O(n^3)$ oder schlechter
 - Dünnbesetzte Matrizen: Direkte Methoden zerstören *sparsity*-Struktur
 - Oft brauchen wir nur wenige Eigenwerte (z.B. die größten $k \ll n$)
- *Warum iterativ?*
 - Schneller wenn nur wenige Eigenwerte benötigt
 - Nutzen *sparsity*-Struktur aus (nur Matrix-Vektor-Multiplikationen!)
 - Trade-off “Genauigkeit vs. Aufwand” ist steuerbar

Potenzmethode

Die **Potenzmethode** (nach Mises) liefert über Iteration den größten (separierten!) Eigenwert λ_1 und Eigenvektor $\mathbf{v}_1$ einer Matrix $\mathbf{A}$:

Idee:

Mit $\mathbf{x}^{(k)} = \frac{\mathbf{A}\mathbf{x}^{(k-1)}}{\|\mathbf{A}\mathbf{x}^{(k-1)}\|}$ gilt

$$\lambda^{(k)} = \|\mathbf{A}\mathbf{x}^{(k)}\| \xrightarrow{k \rightarrow \infty} |\lambda_1| \text{ und } \mathbf{x}^{(k)} \xrightarrow{k \rightarrow \infty} \pm \mathbf{v}_1$$

(Die Norm liefert nur den Betrag $|\lambda_1|$ – das Vorzeichen von λ_1 liefert sie nicht; für negatives λ_1 oszilliert $\mathbf{x}^{(k)}$ zwischen $\pm \mathbf{v}_1$.)

Intuition:

- Für diagonalisierbare $\mathbf{A}$ ist jeder Startvektor Linearkombination der Eigenvektoren $\mathbf{v}_i$ von $\mathbf{A}$:
 $\mathbf{x}^{(0)} = \sum_i^n c_i \mathbf{v}_i$
- Jede Iteration verstärkt die Komponente von $\mathbf{x}^{(k)}$ in Richtung $\mathbf{v}_1$ mehr als die anderen Komponenten

Algebraisch:

Wegen $\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \implies \mathbf{A}^k\mathbf{v} = \lambda^k\mathbf{v}$ gilt auch:

- $\mathbf{A}^k \mathbf{x}^{(0)} = \lambda_1^k \left(\mathbf{c}_1 \mathbf{v}_1 + \sum_{i=2}^n \mathbf{c}_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{v}_i \right) \approx \lambda_1^k \mathbf{c}_1 \mathbf{v}_1$ für $k \rightarrow \infty$ weil

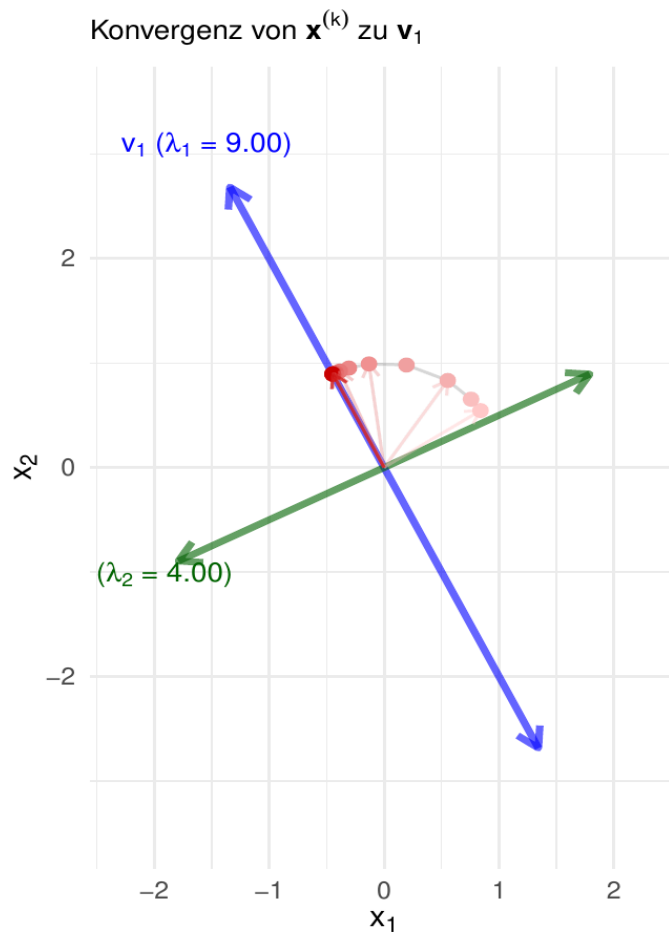
$$\left(\frac{\lambda_i}{\lambda_1} \right)^k \xrightarrow{k \rightarrow \infty} 0 \text{ für } i \geq 2$$

- $\frac{\mathbf{A}^k \mathbf{x}^{(0)}}{\|\mathbf{A}^k \mathbf{x}^{(0)}\|} \approx \frac{\lambda_1^k \mathbf{c}_1 \mathbf{v}_1}{|\lambda_1^k \mathbf{c}_1| \|\mathbf{v}_1\|} = \pm \mathbf{v}_1$

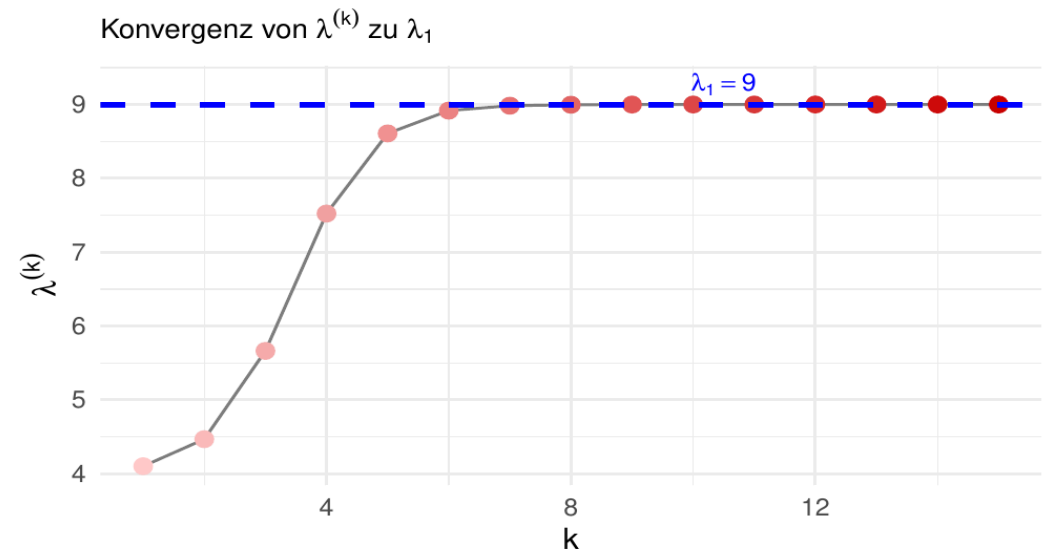
Potenzmethode: Beispiel

$$A = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \mathbf{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \mathbf{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix} \text{ mit } \mathbf{x}^{(0)} = \begin{pmatrix} 2 \\ 1.3 \end{pmatrix}$$

:



k	Winkel zu v[1]
0	83.54°
1	75.71°
2	60.18°
3	37.79°
5	8.71°
10	0.15°
15	0.00°



Quiz

Für welche $\boldsymbol{x}^{(0)}$ versagt die Potenzmethode?

1. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_1$

2. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_2$

3. $\boldsymbol{x}^{(0)} = c(\boldsymbol{v}_1 + \boldsymbol{v}_2)$

Iterative Berechnungsverfahren

- Wir wollen (alle) Eigenwerte/-vektoren von $\mathbf{A} \in \mathbb{R}^{n \times n}$ finden.
- Idee: Überführe $\mathbf{A}$ in spezielle Matrix $\mathbf{B}$, so dass das Problem leichter wird.

Theorem

Sei $\mathbf{B}$ eine zu $\mathbf{A}$ ähnliche Matrix (d.h. $\mathbf{B} = \mathbf{Q}\mathbf{A}\mathbf{Q}^{-1}$). Dann gilt:

- $\mathbf{A}$ und $\mathbf{B}$ haben die gleichen Eigenwerte (und damit auch gleiche Determinante und Spur).
- Ist $\mathbf{y}$ Eigenvektor von $\mathbf{B}$, so ist $\mathbf{x} = \mathbf{Q}^{-1}\mathbf{y}$ Eigenvektor von $\mathbf{A}$ zum gleichen Eigenwert.

Beweis:

$$\mathbf{B}\mathbf{y} = \lambda\mathbf{y} \implies \mathbf{Q}\mathbf{A}\mathbf{Q}^{-1}\mathbf{y} = \lambda\mathbf{y} \implies \mathbf{A}\mathbf{Q}^{-1}\mathbf{y} = \lambda\mathbf{Q}^{-1}\mathbf{y} \implies \mathbf{A}\mathbf{x} = \lambda\mathbf{x}.$$

Beispiel

$$\mathbf{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \mathbf{Q} = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$$

- $\mathbf{Q}$ ist **orthogonal**, also $\mathbf{Q}^{-1} = \mathbf{Q}^\top$: das Beispiel spezialisiert die Ähnlichkeitstransformation $\mathbf{Q}\mathbf{A}\mathbf{Q}^{-1}$ auf *orthogonale* Ähnlichkeit $\mathbf{Q}\mathbf{A}\mathbf{Q}^\top$.

$$\mathbf{B} = \mathbf{Q}(\mathbf{A}\mathbf{Q}^\top) = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

(Rechnung $\rightarrow$ *Anhang*)

- $\mathbf{B}$ diagonal $\implies$ Eigenwerte direkt ablesbar: 4 und 9
- **Der Punkt:**
Ähnliche Matrizen haben dieselben Eigenwerte – $\mathbf{A}$ hat also ebenfalls die Eigenwerte 4 und 9, obwohl man sie an $\mathbf{A}$ nicht ablesen kann.

Verifikation:

$$\det(\mathbf{A} - \lambda \mathbf{I}) = (5 - \lambda)(8 - \lambda) - 4 = \lambda^2 - 13\lambda + 36 = (\lambda - 4)(\lambda - 9) = 0 \quad \checkmark$$

Verfahren

Zur (iterativen) Berechnung einer zu $\mathbf{A}$ ähnlichen Matrix $\mathbf{B}^{(k)}$ deren Eigenwerte einfach zu bestimmen sind gibt es verschiedenste Ideen/Methoden:

- **QR-Iteration** führt zu (approximativer) Dreiecksmatrix $\mathbf{B}$ (*Schur-Zerlegung*).
 - ähnlich für **LU**- und **Cholesky**-Iteration
- *Lanczos-Iteration* führt zu tridiagonalem $\mathbf{B}$ ($b_{ij} = 0$ für $|i - j| > 1$) und kann auf die “wichtigsten” Eigenwerte eingeschränkt werden. Effizienter für große, dünn besetzte Matrizen.
- In der Praxis brauchen wir zusätzliche numerische Tricks, um diese Methoden **stabil** und akzeptabel schnell zu machen: *Shifted QR*, *Implicitly Restarted Lanczos*

QR-Iteration

Def.: QR-Iteration

Die **QR-Iteration** nutzt die *QR-Zerlegung* zur Iteration:

1. Starte mit $\mathbf{A}^{(0)} = \mathbf{A}$
2. Für $k = 1, 2, \dots$:
 - Berechne QR-Zerlegung: $\mathbf{A}^{(k-1)} = \mathbf{Q}^{(k)} \mathbf{R}^{(k)}$
 - Setze $\mathbf{A}^{(k)} = (\mathbf{Q}^{(k)})^\top \mathbf{A}^{(k-1)} \mathbf{Q}^{(k)} = \mathbf{R}^{(k)} \mathbf{Q}^{(k)}$

- $\mathbf{A}^{(k)}$ sind ähnlich zu $\mathbf{A}$:

$$\mathbf{A}^{(k)} = \mathbf{R}^{(k)} \mathbf{Q}^{(k)} = (\mathbf{Q}^{(k)})^{-1} \mathbf{A}^{(k-1)} \mathbf{Q}^{(k)} = (\mathbf{Q}^{(1)} \dots \mathbf{Q}^{(k)})^{-1} \mathbf{A} (\mathbf{Q}^{(1)} \dots \mathbf{Q}^{(k)})$$

- Für $k \rightarrow \infty$ konvergiert $\mathbf{A}^{(k)}$ gegen eine obere Dreiecksmatrix (!) $\rightarrow$ Eigenwerte auf der Diagonalen (für symmetrisches $\mathbf{A}$ mit separierten $|\lambda_i|$; allgemein: Schur-Form)
- Berechnung der Eigenvektoren $\mathbf{V}$ über Produkt $\mathbf{Q}_k := \prod_{j=1}^k \mathbf{Q}^{(j)}$ (für symmetrische $\mathbf{A}$).

Beweis, dass $\mathbf{A}^k = \mathbf{Q}_k \mathbf{R}_k$: siehe **Anhang**

QR-Iteration: Interpretation

- Die QR-Iteration berechnet iterativ die **QR-Zerlegung** von A^k :

$$A^k = Q_k R_k \text{ mit } Q_k := Q^{(1)} \dots Q^{(k)}$$

(Beweis $\rightarrow$ **Anhang**)

- Spalten von Q_k sind orthonormalisierte Versionen von $A^k e_1, A^k e_2, \dots, A^k e_n$
 $\rightarrow$ entspricht **simultaner Potenzmethode** auf **allen** Einheitsvektoren!

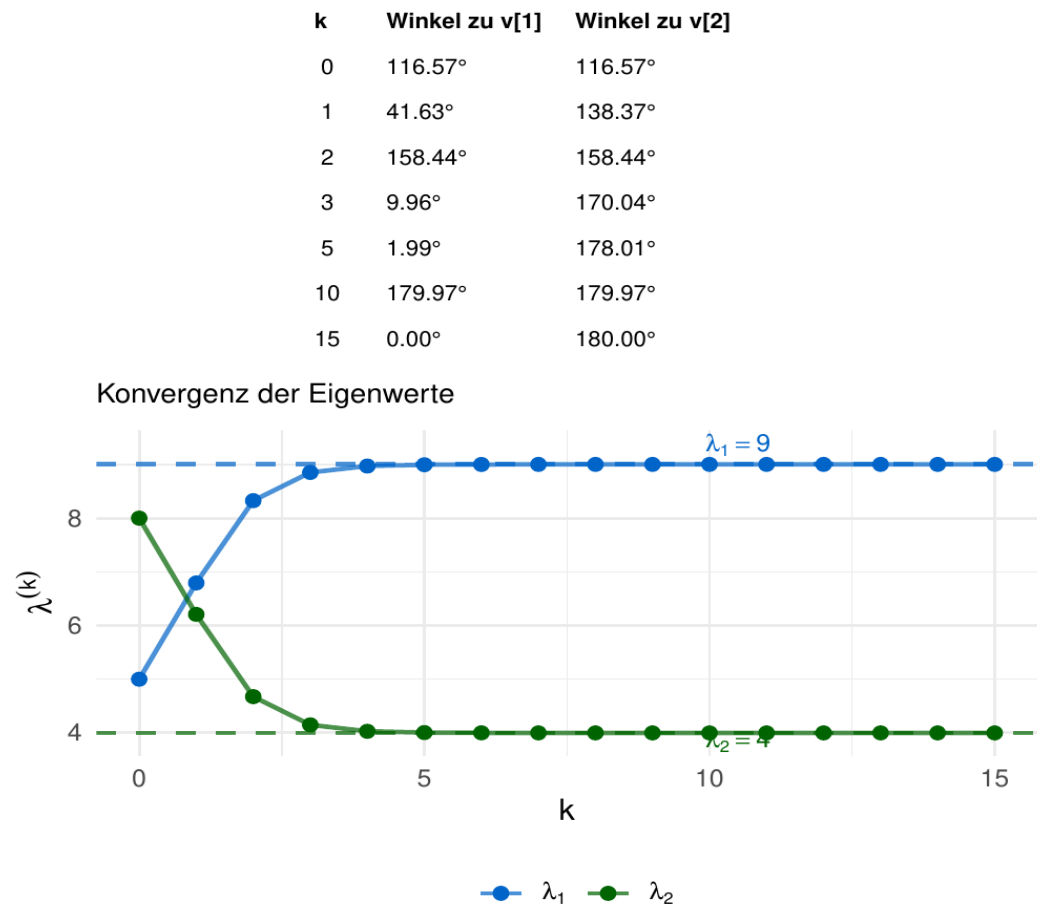
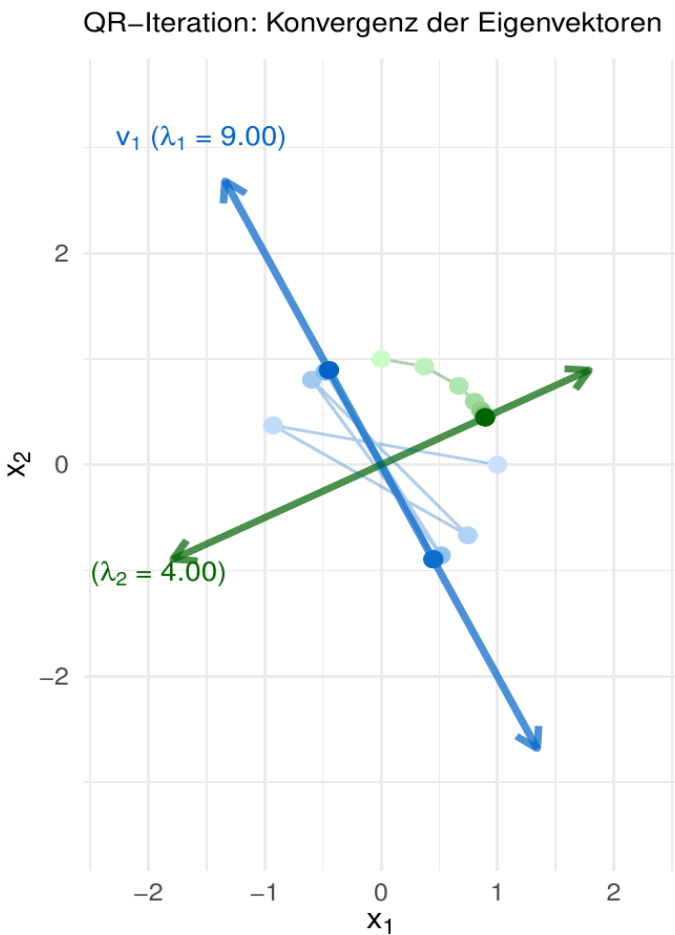
$$\implies Q_k \xrightarrow{k \rightarrow \infty} V$$

- $A^{(k)} = Q_k^\top A Q_k$ konvergiert wegen $Q_k \xrightarrow{k \rightarrow \infty} V$ gegen eine obere Dreiecksmatrix mit Eigenwerten auf der Diagonale (**Schur-Zerlegung**);
für **symmetrische** $A = V \Lambda V^\top$ (mit **orthogonalem** V) sogar gegen die Diagonalmatrix Λ :

$$Q_k^\top A Q_k \xrightarrow{k \rightarrow \infty} V^\top A V = V^\top V \Lambda V^\top V = \Lambda$$

QR-Iteration: Beispiel

$$A = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \mathbf{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \mathbf{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$



Bemerkungen

Konditionierung von Eigenwertproblemen:

- Eigenwertprobleme sind gut *konditioniert*, wenn die Matrix nahezu *symmetrisch* ist.
- Eigenvektorprobleme sind *schlecht konditioniert*, wenn *Eigenwerte nahe beieinander* liegen.

Laufzeit:

- Jede QR-Iteration ist $O(n^3)$
- Insgesamt werden oft nur wenige Iterationen benötigt (abhängig von Eigenwertverteilung).
- Iterative *SVD*-Verfahren (z.B. *Lanczos Bi-Diagonalization*; `{irlba}`) sind noch schneller für große, dünn besetzte Matrizen

Anwendung: Google PageRank

Problem:

Rangordnung von $n \approx 10^{10}$ Webseiten im Internet bilden.

Modell:

Jede Seite i hat einen “Wichtigkeits-Score” x_i , der die Wichtigkeit der Seiten verlinkt auf i gewichtet:

$$x_i = \sum_{j:j \rightarrow i} \frac{x_j}{d_j} \iff \mathbf{x} = \mathbf{A}\mathbf{x} \text{ mit } \mathbf{A} = \left[\frac{I(j \rightarrow i)}{d_j} \right]_{i,j} \implies \text{Eigenwertproblem mit } \lambda = 1!$$

(“ $j \rightarrow i$ ” heißt “Seite j verlinkt auf i ”, d_j sei Anzahl ausgehender Links von j)

(Setzt voraus, dass jede Seite mindestens einen ausgehenden Link hat – sonst ist $d_j = 0$ und die Spalte j gar nicht definiert; solche Spalten füllt man gleichverteilt auf.)

Frage: Warum kann Google keine $10^{10} \times 10^{10}$ -Matrix invertieren – und warum funktioniert die Eigenwert-Berechnung trotzdem?

- Direkte Methoden unmöglich: $\mathbf{A}$ dicht gespeichert wären 10^{20} Einträge (≈ 800 Exabyte), Inversion/Zerlegung $O(n^3) = 10^{30}$ Operationen.
- Aber: $\mathbf{A}$ ist *extrem dünn besetzt* (jede Seite hat nur ~ 10 Links) $\implies$ Speicherbedarf $O(n)$ statt $O(n^2)$
- **Potenzmethode:** $\mathbf{x}^{(k+1)} = \mathbf{A}\mathbf{x}^{(k)} / \|\mathbf{A}\mathbf{x}^{(k)}\|$ braucht nur Matrix-Vektor-Produkte: eine Iteration ist $O(n)$, Konvergenz in ~ 50 Iterationen
- **Gesamt:** $O(n)$ statt $O(n^3)$ – macht Google überhaupt erst möglich!

Anwendung: Hauptkomponentenanalyse (PCA)

Berechnung von Eigenwerten/-vektoren für Datenanalysen essentiell:

- **Problem:** Gegeben ist eine mittelwert-zentrierte Datenmatrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ (n Beobachtungen, p Variablen)
- **Ziel:** Finde Richtungen maximaler Varianz zur Dimensionsreduktion
- **Lösung:** Berechne die Kovarianzmatrix $\mathbf{\Sigma} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{p \times p}$
 - Eigenvektoren $\mathbf{v}_j$ von $\mathbf{\Sigma}$ sind die *Hauptachsen* bzw. *Loading-Richtungen* maximaler Varianz in $\mathbb{R}^p$
 - Die *Hauptkomponenten* (Scores) sind $\mathbf{z}_j = \mathbf{X} \mathbf{v}_j$; die Eigenwerte geben ihre Varianz an
- In der Praxis noch schneller/effizienter über *SVD* von $\mathbf{X}$ (z.B. `{irlba}: irlba()` (iterativ), `svdr()` (iterativ & probabilistisch))

Warum iterative Methoden?

- Moderne Anwendungen: $p = 10^4$ bis 10^6 (z.B. Gen-Expression, Bildanalyse, ...)
- Oft nur die k größten Eigenwerte benötigt ($k \ll p$) - vgl. *Bildkompression* durch *Reduzierte SVD*
- Lanczos-Iteration: Berechnet nur die k größten Eigenwerte in $O(kp^2)$ statt $O(p^3)$ für alle
 - Bei $p = 100.000$ und $k = 10$ ist Lanczos ~ 10.000 mal schneller!

Anwendung: Approximative SVD

Bildkompression durch **Reduzierte SVD**: X ist ein Graustufen-Foto als 2500×3300 -Matrix (Setup-Code, Fehlervergleich & Bildpanel $\rightarrow$ **Anhang**):

```
1 r <- 50
2 microbenchmark::microbenchmark(
3   svd_full    = svd(X),                # exakt: alle Singulaerwerte
4   svd_irlba   = irlba::irlba(X, nv = r, nu = r), # iterativ (Lanczos): nur r
5   svd_random  = irlba::svdr(X, k = r),      # iterativ & randomisiert
6   times = 5)
```

Unit: seconds

	expr	min	lq	mean	median	uq	max	neval	cld
svd_full		84.14	92.85	107.77	112.60	114.01	135.23	5	a
svd_irlba		1.98	2.09	2.85	2.13	3.46	4.58	5	b
svd_random		5.79	7.00	9.36	10.44	10.77	12.80	5	b

```
1 X_svdr <- with(irlba::svdr(X, k = r), u %*% diag(d) %*% t(v))
2 norm(X - X_svdr, "F") / norm(X, "F") # rel. Fehler der Rang-50-Approximation
```

[1] 0.129

$\rightarrow$ iterative/randomisierte Verfahren sind hier $\sim 40\times$ bzw. $\sim 11\times$ schneller — bei praktisch identischem Ergebnis (Vergleich mit exakter Rang-50-SVD $\rightarrow$ **Anhang**).

Iterative Eigenwertverfahren: Vergleich

Methode	Komplexität / Iteration	Anwendung	Output
Potenz- methode	$O(\text{nnz}(\mathbf{A}))$	Größter Eigenwert/-vektor	1 Eigenpaar
(shifted) QR	$O(n^2)$ - $O(n^3)$	Alle Eigenwerte	n Eigenwerte
Lanczos	$O(m \text{ nnz}(\mathbf{A}) + m^2)$ für m EW	Wenige Extremal-EW, <i>sparse</i> $\mathbf{A}$	$m \ll n$ Eigenpaare

- Konvergenzrate **Potenzmethode**: $|\lambda_2/\lambda_1|$
- **QR-Iteration** mit Shifts oft mit quadratischer Konvergenz, am stabilsten (ohne Shifts nur linear, mit Rate $|\lambda_i/\lambda_j|$).
- Anwendung:
 - *sparse* Matrizen: Lanczos klar überlegen
 - Alle Eigenwerte, Speicher sekundär: (shifted) QR-Iteration
 - Wenige Eigenwerte: Potenzmethode (für 1), Lanczos (für $m \ll n$)

$\text{nnz}(\cdot)$ = Anzahl Nicht-Null-Einträge

Iterative Lösungsverfahren

Iterative Lösungsverfahren: Intuition

- Zur Lösung von **LGS** und **KQ-Problemen** haben wir bisher direkte Methoden gesehen — **iterative Verfahren** (meist **Fixpunktiterationen**) sind oft viel schneller, wenn man etwas Genauigkeit opfert.

Wie nähern wir uns iterativ der Lösung?

- **Residuum:**
 $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$ misst, wie stark $\mathbf{x}^{(k)}$ die Gleichung $\mathbf{A}\mathbf{x} = \mathbf{b}$ verletzt
- Falls $\mathbf{A}\mathbf{x}^{(k)} = \mathbf{b}$, dann ist $\mathbf{r}^{(k)} = \mathbf{0}$ und wir haben die Lösung gefunden
- **Korrektur:**
Solange $\mathbf{r}^{(k)} \neq \mathbf{0}$, korrigieren wir unsere Schätzung:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \underbrace{\mathbf{C}\mathbf{r}^{(k)}}_{\text{Korrekturschritt}}$$

- $\mathbf{C}$ bestimmt, wie stark wir wo in Richtung des Residuums korrigieren
- **Analogie:** *Gradient Descent* - wir bewegen uns Schritt für Schritt zur Lösung

Warum nicht direkt lösen?

→ Bei großen und/oder dünnbesetzten Matrizen sind iterative Methoden oft um Größenordnungen schneller als direkte Verfahren!

Iterative Lösungsverfahren für LGS

- Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$ und $\mathbf{x}$ die Lösung des LGS $\mathbf{Ax} = \mathbf{b}$.
- Für einen beliebigen Startwert $\mathbf{x}^{(0)} \in \mathbb{R}^n$ und $\mathbf{C} \in \mathbb{R}^{n \times n}$, definiere die Iteration

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \mathbf{C}(\mathbf{b} - \mathbf{Ax}^{(k-1)}), \quad k = 1, 2, \dots$$

Theorem

Sei $\mathbf{x}$ die eindeutige Lösung des LGS $\mathbf{Ax} = \mathbf{b}$.

Falls $\rho := \|\mathbf{I}_n - \mathbf{CA}\|_2 < 1$ (Spektralnorm), gilt für alle $k \geq 1$:

$$\|\mathbf{x}^{(k)} - \mathbf{x}\|_2 \leq \rho^k \|\mathbf{x}^{(0)} - \mathbf{x}\|_2.$$

- Insbesondere: $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}$ (Konvergenz).

Beweis: siehe [Anhang](#)

Quiz

Sei

$$\mathbf{x}^{(k)} = (\mathbf{I} - \mathbf{C}\mathbf{A})\mathbf{x}^{(k-1)} + \mathbf{C}\mathbf{b}$$

mit $\mathbf{C} = \gamma\mathbf{I}_n$ für ein $\gamma > 0$. Was ist die Laufzeitkomplexität eines Iterationsschrittes?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Lösung: 3. – Mit $\mathbf{C} = \gamma\mathbf{I}$ ist $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} - \gamma(\mathbf{A}\mathbf{x}^{(k-1)}) + \gamma\mathbf{b}$: pro Schritt eine dichte Matrix-Vektor-Multiplikation $\mathbf{A}\mathbf{x}^{(k-1)}$, also $O(n^2)$; Skalierung mit γ und die zwei Vektoradditionen sind nur $O(n)$.

Iterative Lösungsverfahren für LGS: Wahl von C

- Die Frage ist nun, wie wir C so wählen, dass:
 - $\rho = \|I - CA\|_2 < 1$ möglichst klein ist (optimal: $C \approx A^{-1}$),
 - $x^{(k)} = (I - CA)x^{(k-1)} + Cb$ einfach zu berechnen ist.

Beliebte Wahlen für C mit Iterationsaufwand $O(n^2)$:

- *Richardson*-Iteration: $C = \gamma I_n$; konvergiert z.B. für SPD A , falls $0 < \gamma < 2/\lambda_{\max}(A)$.
- *Jacobi*-Iteration: $C = \text{diag}(A_{11}, \dots, A_{nn})^{-1}$.
- *Gauss-Seidel*-Iteration: C^{-1} gleich unterem Dreieck von A .

Gesamtaufwand:

- Wegen $\|x^{(k)} - x\| \leq \rho^k \|x^{(0)} - x\|$ reichen $k = O(\log(1/\epsilon))$ Schritte für Genauigkeit ϵ ($\rightarrow$ Self-Check).
- Damit brauchen wir $O(n^2 \log(1/\epsilon))$ Operationen für $\|x^{(k)} - x\| \leq \epsilon$.
- Vergleiche: $O(n^3)$ für exakte Lösungen durch **LU** oder **QR**.

Analoge iterative Verfahren gibt es auch für **KQ**-Probleme; in seltenen Fällen sind sie sogar *exakt*. Meist gilt: Trade-off zwischen **Laufzeit** und Genauigkeit – steuerbar über die Iterationszahl.

Bsp: Richardson-Iteration

Gegeben: $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$. Wahre Lösung: $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \frac{1}{11} \begin{pmatrix} 1 \\ 7 \end{pmatrix} \approx \begin{pmatrix} 0.091 \\ 0.636 \end{pmatrix}$.

Richardson mit $\gamma = 0.25$, Startwert $\mathbf{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$:

Iteration 0: $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + 0.25\mathbf{r}^{(0)} = \begin{pmatrix} 0.25 \\ 0.5 \end{pmatrix}$

Iteration 1: $\mathbf{A}\mathbf{x}^{(1)} = \begin{pmatrix} 1.5 \\ 1.75 \end{pmatrix}$, $\mathbf{r}^{(1)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(1)} = \begin{pmatrix} -0.5 \\ 0.25 \end{pmatrix}$, $\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + 0.25\mathbf{r}^{(1)} = \begin{pmatrix} 0.125 \\ 0.563 \end{pmatrix}$

Iteration 2: $\mathbf{A}\mathbf{x}^{(2)} = \begin{pmatrix} 1.063 \\ 1.813 \end{pmatrix}$, $\mathbf{r}^{(2)} = \begin{pmatrix} -0.063 \\ 0.188 \end{pmatrix}$, $\mathbf{x}^{(3)} = \begin{pmatrix} 0.109 \\ 0.609 \end{pmatrix}$

Iteration 3: $\mathbf{x}^{(4)} = \begin{pmatrix} 0.098 \\ 0.625 \end{pmatrix}$

Fehler: $\|\mathbf{x}^{(k)} - \mathbf{x}\|_2: 0.643 \rightarrow 0.210 \rightarrow 0.081 \rightarrow 0.033 \rightarrow 0.013 \rightarrow \dots$

Konvergenzrate: $\rho = \|\mathbf{I} - \gamma\mathbf{A}\|_2 \approx 0.41 \implies$ Fehler sollte ca. um Faktor 2.5 pro Iteration sinken.

Probabilistische Methoden

Zufall mit Absicht

- Oft sind Probleme so groß, dass wir gerne Genauigkeit für bessere *Komplexität* tauschen.
- Heute wird das zunehmend durch *Randomisierung* erreicht.
- **Hauptidee:**
 - Ersetze großes Problem durch ein
 - zufällig erzeugtes, *kleines* Problem,
 - das *höchstwahrscheinlich* eine *ähnliche Lösung* hat.

Idee: Matrix-Sketching

Geometrische Fakten in hoch-dimensionalem Raum $\mathbb{R}^n$ mit $n \gg 1$:

Für großes n sind zwei zufällige Vektoren $\mathbf{s}_1, \mathbf{s}_2 \in \mathbb{R}^n$ mit hoher Wahrscheinlichkeit fast orthogonal.

$\implies$ geeignet zufällig gezogene Matrix $\mathbf{S} \in \mathbb{R}^{m \times n}$ mit $m \ll n$ erhält mit hoher Wahrscheinlichkeit

- Abstände: $\|\mathbf{S}\mathbf{x} - \mathbf{S}\mathbf{y}\|_2 \approx \|\mathbf{x} - \mathbf{y}\|_2$
- Winkel: $\angle(\mathbf{S}\mathbf{x}, \mathbf{S}\mathbf{y}) \approx \angle(\mathbf{x}, \mathbf{y})$

$\implies$ Eine **zufällige Projektion** $\mathbf{S}$ in einen niedrigdimensionaleren Raum kann Geometrie oft näherungsweise erhalten.

Beispiel: Sketching in R

```
1 set.seed(1312)
2 n <- 10000          # zwei beliebige n-dimensionale Vektoren:
3 x <- runif(n); y <- runif(n)
4 m <- 50             # m << n !!
5 S <- rnorm(m * n, sd = sqrt(1 / m)) |> matrix(nrow = m, ncol = n)
6 Sx <- S %%% x; Sy <- S %%% y
```

Wir ersetzen $n = 10000$ Koordinaten durch $m = 50$ zufällige Projektionen.

Vorhersage:

Um wie viel Prozent ändern sich Distanz $\|\mathbf{x} - \mathbf{y}\|_2$ und Winkel $\angle(\mathbf{x}, \mathbf{y})$ dabei wohl — eher $\sim 50\%$, $\sim 3\%$ oder $< 0.1\%$?

```
1 angle <- function(x, y) (sum(x * y) / (norm(x, "2") * norm(y, "2"))) |> acos()
2 c(norm(x - y, "2"), norm(Sx - Sy, "2")) ## Distanzen ||x - y||, ||Sx - Sy||
3 c(angle(x, y), angle(Sx, Sy))          ## Winkel (rad)
```

```
[1] 41.0 42.2
[1] 0.728 0.735
```

→ nur $\approx 3\%$ [$\approx 1\%$] Abweichung in Distanz [Winkel] — obwohl 99.5% der Dimensionen wegfallen!

(Eine einzelne Ziehung, und eine recht glückliche: für Gauss-Skizzen streut die relative Abweichung der Distanz mit $\approx 1/\sqrt{2m} = 10\%$.)

Zufällige Einbettung eines festen Vektors

Theorem

Sei

- $\mathbf{S} \in \mathbb{R}^{m \times n}$ eine Matrix mit i.i.d. Zeilen $\mathbf{s}_i^\top$ (wobei $\mathbf{s}_i \in \mathbb{R}^n$),
- $\mathbb{E}[\mathbf{s}_i \mathbf{s}_i^\top] = \mathbf{I}_n/m$ und $\mathbb{E}[(\mathbf{s}_i^\top \mathbf{x})^4] \leq K \|\mathbf{x}\|^4/m^2$ für alle $\mathbf{x} \in \mathbb{R}^n$.

Dann gilt für jeden festen Vektor $\mathbf{x} \in \mathbb{R}^n$ mit einer Wahrscheinlichkeit von mindestens $1 - \delta$

$$(1 - \epsilon) \|\mathbf{x}\|^2 \leq \|\mathbf{S}\mathbf{x}\|^2 \leq (1 + \epsilon) \|\mathbf{x}\|^2,$$

wobei $\delta > 0$ und $\epsilon = \sqrt{K/\delta m}$.

Beispiele:

- *Gauss*: $\mathbf{s}_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \mathbf{I}_n/m)$.
- *Rademacher*: s_{ij} i.i.d. mit $\mathbb{P}(s_{ij} = 1/\sqrt{m}) = \mathbb{P}(s_{ij} = -1/\sqrt{m}) = 1/2$.
- *Subsampling*: $\mathbf{s}_i = \sqrt{n/m} \mathbf{e}_{K_i}$ skaliertes Einheitsvektor mit $K_i \sim \text{Unif}\{1, \dots, n\}$.

Beweis: siehe **Anhang**

Anwendungen

- Zufällige Einbettungen (*Matrix-Sketching*) haben zahlreiche Anwendungen:
 - Matrixmultiplikation: $\mathbf{AB} \approx (\mathbf{AS}^\top)(\mathbf{SB})$
 - *KQ-Probleme*: $\min \|\mathbf{Ax} - \mathbf{b}\|_2 \approx \min \|\mathbf{SAx} - \mathbf{Sb}\|_2$
 - ...
- *Achtung*: Unser Theorem gilt je festem Vektor — hier hängt z.B. der Minimierer selbst von $\mathbf{S}$ ab! Dafür braucht man stärkere, *simultane* Garantien (*Subspace Embeddings*): gleiche Beweisidee, etwas größeres m .
- Wirklich schneller werden Algorithmen aber nur, wenn wir Produkte $\mathbf{Sx}$ sehr schnell berechnen können.
- *Beispiele*:
 - Wenn $\mathbf{S}$ m zufällige Koordinaten aus $\mathbf{x}$ zieht, braucht es dafür nur $O(m)$ Operationen.
 - Besonders leistungsfähig: *Subsampled Randomized Hadamard Transform* (SRHT) braucht $O(n \log m)$ Rechenschritte. (Vergleich der Sketching-Matrizen → *Anhang*)

Beispiel: Dimensionsreduktion mit Matrix-Sketching

Gegeben:

$n = 10000$ Dimensionen, $\delta = 0.05$, $\epsilon = 0.1$

Frage:

Wie klein kann m sein?

Aus dem Theorem:

Mit $\mathbf{s}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n/m)$ gilt $K = 3$ und

$$\epsilon = \sqrt{\frac{K}{\delta m}} \implies m = \frac{K}{\delta \epsilon^2} = \frac{3}{0.05 \cdot 0.01} = 6000$$

Ergebnis:

Mit $m = 6000$ Dimensionen können wir $n = 10000$ Dimensionen so darstellen, dass jeder gegebene Abstand mit 95% Wahrscheinlichkeit um höchstens 10% verzerrt wird.

Praktische Konsequenz:

- Speicher & Rechenzeit pro Distanzberechnung: $O(n) \rightarrow O(m)$
- Die Schranke (via Tschebyscheff-Ungleichung) ist sehr *konservativ* — in der Praxis reichen oft viel kleinere m : die Demo oben kam mit $m = 50$ auf nur $\approx 3\%$ Abweichung!
- Anwendung: Schnelle Nächste-Nachbar-Suche, Clusteranalyse in hohen Dimensionen, ...

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Lösungsansätze für Eigenwertprobleme
- *Iterative Verfahren* nähern sich Lösung schrittweise durch *Fixpunktiteration*
- *Probabilistische Verfahren* ersetzen schweres durch zufälliges, leichteres (niedrigdimensionaleres) Problem mit approximativer Lösung
- Beide geben uns einen Trade-off zwischen *Laufzeit* und Genauigkeit

Nächste Vorlesung

- Tensoren und Tensorprodukte

Self-Check (1/2)

Frage 1: Sei $\rho \in (0, 1)$, $\epsilon > 0$, und

$$\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \rho^k \|\mathbf{x}^{(0)} - \mathbf{x}\|.$$

Wie groß muss k mindestens sein, so dass $\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \epsilon$?

1. $k = O(1)$
2. $k = O(\log(1/\epsilon))$
3. $k = O(1/\epsilon)$
4. $k = O(1/\epsilon^k)$

Lösung: 2. – mit $e_0 = \|\mathbf{x}^{(0)} - \mathbf{x}\|$: $\rho^k e_0 \leq \epsilon \implies \rho^k \leq \epsilon/e_0$; Logarithmieren ($0 < \rho < 1 \implies \log \rho < 0$!) gibt $k \geq \frac{\log(\epsilon) - \log(e_0)}{\log(\rho)} = \frac{\log(e_0) + \log(1/\epsilon)}{-\log(\rho)}$, also minimales $k = \left\lceil \frac{\log(e_0/\epsilon)}{-\log(\rho)} \right\rceil$ (falls $0 < \epsilon < e_0$, sonst reicht schon $k = 0$) – wächst wie $\log(1/\epsilon)$. ✓

Self-Check (2/2)

Frage 2: Worin unterscheidet sich *Subsampling* (m zufällige, skalierte Zeilen) von einer *Gauss-Sketching-Matrix* vor allem?

1. Subsampling kostet nur $O(m)$ statt $O(mn)$ (Zeit & Speicher), ist aber nur zuverlässig, wenn die Information über viele Koordinaten gestreut ist.
2. Gauss-Matrizen brauchen weniger Speicher als Subsampling.
3. Subsampling erhält alle Distanzen exakt.
4. Es gibt keinen Unterschied — beide sind optimal und gleich teuer.

Frage 3: Welcher Sketching-Typ kombiniert schnelle Anwendung ($O(n \log m)$ für Sx) mit sehr guter Qualität (“State of the art”)?

1. Gauss
2. Rademacher
3. Subsampling
4. Subsampled Randomized Hadamard Transform (SRHT)

Anhang

Ähnlichkeit: Beispiel – Rechnung

$$\mathbf{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \mathbf{Q} = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$$

Schritt 1: Berechne $\mathbf{A}\mathbf{Q}^\top$:

$$\mathbf{A}\mathbf{Q}^\top = \frac{1}{\sqrt{5}} \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} = \frac{1}{\sqrt{5}} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix}$$

Schritt 2: Berechne $\mathbf{B} = \mathbf{Q}(\mathbf{A}\mathbf{Q}^\top)$:

$$\mathbf{B} = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 20 & 0 \\ 0 & 45 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

$\mathbf{B}$ diagonal $\implies$ Eigenwerte direkt ablesbar: 4 und 9.

↩ zurück: „Beispiel“

QR-Iteration: Beweis

Definiere $Q_k := Q^{(1)} Q^{(2)} \dots Q^{(k)}$, $R_k = R^{(k)} R^{(k-1)} \dots R^{(1)}$

Zeige: $A^k = Q_k R_k$

Beweis:

- Induktionsanfang ($k = 1$): $A = Q^{(1)} R^{(1)} = Q_1 R_1 \checkmark$
- Induktionsschritt ($k \rightarrow k + 1$): $A^{(i)} = R^{(i)} Q^{(i)}$,
 $A^{(i)} = Q^{(i+1)} R^{(i+1)} \implies R^{(i)} Q^{(i)} = Q^{(i+1)} R^{(i+1)} \quad (\star)$

$$\begin{aligned} A^{k+1} &= A A^k = (Q^{(1)} R^{(1)}) (Q_k R_k) \\ &= Q^{(1)} (R^{(1)} Q^{(1)}) Q^{(2)} \dots Q^{(k)} R^{(k)} \dots R^{(1)} \\ &\quad [\text{nutze } k\text{-mal } (\star) : R^{(1)} Q^{(1)} = Q^{(2)} R^{(2)}, \dots] \\ &= Q^{(1)} Q^{(2)} \dots Q^{(k+1)} R^{(k+1)} \dots R^{(1)} \\ &= Q_{k+1} R_{k+1} \quad \checkmark \end{aligned}$$

↔ zurück: „QR-Iteration” · ↔ zurück: „QR-Iteration: Interpretation”

Iterative LGS-Verfahren: Konvergenz — Beweis

Definiere $B = I - CA$, sodass $\|B\|_2 = \rho$ und

$$x^{(k)} = Bx^{(k-1)} + Cb, \quad k = 1, 2, \dots$$

Zunächst bemerken wir, dass x ein Fixpunkt dieser Iteration ist:

$$Bx + Cb = (I - CA)x + Cb = x - \underbrace{C(Ax - b)}_{=0} = x$$

also

$$\begin{aligned} \|x^{(k)} - x\|_2 &= \|Bx^{(k-1)} + Cb - x\|_2 \\ &= \|Bx^{(k-1)} + Cb - (Bx + Cb)\|_2 \\ &= \|B(x^{(k-1)} - x)\|_2 \\ &\leq \rho \|x^{(k-1)} - x\|_2. \end{aligned}$$

Iterieren wir die Ungleichung, erhalten wir

$$\|x^{(k)} - x\|_2 \leq \rho \|x^{(k-1)} - x\|_2 \leq \rho^2 \|x^{(k-2)} - x\|_2 \leq \dots \leq \rho^k \|x^{(0)} - x\|_2.$$



↩ zurück: „Iterative Lösungsverfahren für LGS“

Zufälliges Einbetten von Unterräumen: Beweis (1/2)

1. Für $\mathbf{x} = \mathbf{0}$ ist die Aussage trivial. Sei also $\mathbf{x} \neq \mathbf{0}$ und o.B.d.A. $\|\mathbf{x}\| = 1$ (andernfalls teile die Ungleichung durch $\|\mathbf{x}\|^2$). Zu zeigen also

$$\mathbb{P} \left(1 - \sqrt{K/\delta m} \leq \|\mathbf{S}\mathbf{x}\|^2 \leq 1 + \sqrt{K/\delta m} \right) = \mathbb{P} \left(|\|\mathbf{S}\mathbf{x}\|^2 - 1| \leq \sqrt{K/\delta m} \right) \geq 1 - \delta$$

2. Mit $\|\mathbf{S}\mathbf{x}\|^2 = \sum_{i=1}^m (\mathbf{s}_i^\top \mathbf{x})^2$ gilt

$$\mathbb{E} [\|\mathbf{S}\mathbf{x}\|^2] = \sum_{i=1}^m \mathbb{E} [(\mathbf{s}_i^\top \mathbf{x})^2] = m \mathbf{x}^\top \mathbb{E} [\mathbf{s}_1 \mathbf{s}_1^\top] \mathbf{x} = \|\mathbf{x}\|^2.$$

3. Weil die Summanden unabhängig sind, gilt außerdem

$$\text{Var} [\|\mathbf{S}\mathbf{x}\|^2] = \sum_{i=1}^m \text{Var} [(\mathbf{s}_i^\top \mathbf{x})^2] \leq m \mathbb{E} [(\mathbf{s}_1^\top \mathbf{x})^4] \leq \frac{K}{m}$$

nach der Momentannahme (mit $\|\mathbf{x}\| = 1$).

Zufälliges Einbetten von Unterräumen: Beweis (2/2)

(Fortsetzung)

4. Nun gibt die Tschebyscheff-Ungleichung (= Markov, angewandt auf die quadrierte Abweichung)

$$\mathbb{P} \left(\left| \|\mathbf{S}\mathbf{x}\|^2 - \mathbb{E}[\|\mathbf{S}\mathbf{x}\|^2] \right| > \epsilon \right) \leq \frac{\text{Var}[\|\mathbf{S}\mathbf{x}\|^2]}{\epsilon^2} \leq \frac{m \mathbb{E}[(\mathbf{s}_i^\top \mathbf{x})^4]}{\epsilon^2} \leq \frac{K}{\epsilon^2 m}.$$

Die Wahl $\epsilon = \sqrt{K/\delta m}$ liefert die Behauptung. ■

↪ zurück: „Zufälliges Einbetten von Unterräumen“

Approximative SVD: Setup & Fehlervergleich

Setup-Code des Benchmarks (im Foliensatz ausgeführt):

```
1 set.seed(1312)
2 library(irlba)
3 suppressMessages(library(magick))
4 img <- paste0(
5   "https://upload.wikimedia.org/wikipedia/commons/7/7e/",
6   "Bolzano_City_Image_-_Photo_by_Giovanni_Ussi_-_In_Black_and_White_19.jpg")
7 X <- (image_read(img) |>
8   image_data(channels = "gray") |>
9   as.numeric())[,,1]
```

Relative Frobenius-Fehler der niedrigrangigen Approximationen (der exakten Rang-50-SVD zu $\mathbf{X}$, und der iterativen Verfahren zur exakten Rang-50-SVD):

```
1 X_r <- with(svd(X, nu = r, nv = r), u %*% diag(d[1:r]) %*% t(v))
2 X_irlba <- with(irlba::irlba(X, nu = r, nv = r), u %*% diag(d) %*% t(v))
3 norm(X - X_r, "F") / norm(X, "F")
```

```
[1] 0.129
```

```
1 c(norm(X_r - X_irlba, "F"), norm(X_r - X_svdr, "F")) / norm(X_r, "F")
```

```
[1] 2.27e-06 1.68e-04
```

→ `irlba/svdr` weichen von der exakten Rang-50-SVD um Größenordnungen weniger ab, als diese vom Originalbild.

Approximative SVD: Bildvergleich

Original



SVD ($r = 50$)



IRLBA ($r = 50$)



SVDR ($r = 50$)



↔ zurück: „Anwendung: Approximative SVD“

Vergleich Sketching-Matrizen

Typ	Speicher	Zeit für Sx	Qualität
Gauss	$O(mn)$	$O(mn)$	Optimal
Rademacher ($\pm 1/\sqrt{m}$)	$O(mn)$	$O(mn)$	Optimal
Subsampling	$O(m)$	$O(m)$	Gut (bei über viele Koordinaten <i>gestreuter</i> Information)
SRHT (Hadamard)	$O(n)$	$O(n \log m)$	Sehr gut

Trade-offs:

- **Gauss:** Theoretisch optimal, aber teuer zu speichern und anzuwenden
- **Rademacher:** Genauso gut wie Gauss, einfacher zu generieren (nur ± 1)
- **Subsampling:** Wählt m zufällige Zeilen - extrem schnell, aber nur gut, wenn die Information über viele Koordinaten gestreut ist (schlecht, wenn sie in wenigen Koordinaten konzentriert ist)
- **SRHT:** kombiniert hohe Geschwindigkeit und gute Approximationsqualität; Speicher $O(n)$ wegen der n Vorzeichen der vorgeschalteten Diagonalmatrix (sofern man sie nicht aus einem Seed neu erzeugt)
 - *Subsampled Randomized Hadamard Transform*
 - S wird nicht explizit gespeichert, sondern durch schnelle(!) Transformationen angewendet